Unit 05

EXPERIMENTATION AND CODING IN SCIENTIFIC RESEARCH

1. **EXPERIMENTATION**

EXPERIMENTATION

- The use of experiments to verify hypotheses is one of the central elements of science.
- In computing, experiments are used for purposes such as confirming hypotheses about algorithms and systems.
- For example: An experiment can verify that a system can complete a specified task and can do so with reasonable use of resources.
- A tested hypothesis becomes part of scientific knowledge if it is sufficiently well described and constructed, and if it is convincingly demonstrated.

EXPERIMENTATION

- Experiments in computing take diverse forms, from tests of algorithm performance to human factors analysis.
- Tests should be fair rather than constructed to support the hypothesis.
- If the design of tests seems biased towards the intended contribution, readers will not be persuaded by the results.
- Today topic: the design, execution, and description of experiments in computing

BASELINES

- An essential step in the design of experiments is to **identify an appropriate *baseline***.
- For example, no sensible researcher would advocate that their new sorting algorithm was a breakthrough on the basis that it is faster than bubblesort; instead, the algorithm should be compared to the best previous method.
- A benchmark is only compelling if it is implemented to a **high standard**, and thus it may be that comparison to a baseline is difficult.
- Without such a comparison it may be impossible for the reader to know whether the new method offers an improvement.

BASELINES

- A problem in an ongoing research program can arise when a well-known, widely available implementation becomes commonly used as a reference point.
- When the worth of every new contribution is shown by comparison to the same baseline system (or, in some cases, baseline data set), in some respects the field benefits, because the use of the common resource means that readers can have confidence that the baseline is accurate.
- However, in other respects the field may suffer, because the advances that are being described may not be cumulative.

BASELINES

- It is critical that baselines be **identified early** in the research program.
  - What is the point of developing new methods if existing methods—or, perhaps worse, trivial methods—provide a satisfactory solution?
  - In work with Web data, for example, we found that problems we were experiencing with parsing of the text might in principle be resolved by automatically determining which (European) language each page was written in. To our surprise, a trivial method based on counting occurrences of a small number of representative words (such as “the” for English or “der” for German) gave 100% accuracy on our test data. Plans to investigate richer techniques had to be abandoned.

PERSUASIVE DATA

- For work that involves experiments, it is critical that you have access to appropriate data, and that you understand it well. You need to consider:
  - What data may be available, and whether it is created by you or sourced from elsewhere.
  - What specific mechanisms will be used to gather and standardize the data.
  - Whether the data will be sufficient in volume or quality to give a robust answer to the question.
  - What domain knowledge may be required to properly interpret the data.
  - What the limits, biases, flaws, and properties of the data are likely to be, and how these problems will be addressed or managed.
  - What the results will be like if the data supports the hypothesis; or, alternatively, what they will be like if the hypothesis is false.

EXAMPLE

- Consider a detailed example.
  - A *microarray* is a technology that can be used to cheaply obtain the genetic profile of a human, by identifying, for each of say a million common genetic variations, whether the variation is present or absent.
  - There is a large collection of individuals that have a microarray profile, and other *phenome* data on the individuals, such as health status or physical characteristics.
  - We can analyze the profiles to see if there is a clear link from specific genetic variations to specific aspects of the phenome,
    - For example to try and identify variations that are linked to occurrences of a particular cancer.

DATA AND MECHANISMS

- ***What data***:
  - The researcher could collect data directly, but it is much cheaper to obtain data from a public database / dataset.
  - Such data will often be associated with previous publications, so their characteristics should be well understood.
- ***What mechanisms***:
  - A good approach to choose datasets could be to find research papers that draw important conclusions based on particular data of the right kind, and then obtain that data.
  - The data may then need to be normalized or cleaned up in some defensible way: if the data was originally gathered for other purposes, some of it may not be suitable for the current investigation; or it may be in an inconsistent format; …

SUFFICIENCY OF DATA

- ***Sufficiency of data***: There are several respects in data volume is relevant:
  - Algorithmic: methods tend to behave differently at different scales.
  - Statistic significance; data volumes need to be large enough to ensure that the experiment will be able to detect the effect that is being hypothesized.
  - Sufficiency also has another dimension—the number of data sets being used.
    - A single data set may not be persuasive, particularly if the reader suspects that the method was tuned to perform well on the data set reported in the paper.

DOMAIN KNOWLEDGE

- ***Domain knowledge***:
  - A failure in some computer science research is that a real-world problem has been abstracted or simplified in some way that makes it unrealistic, and thus the methods that are developed have no practical relevance.
  - For example, there would be little value to a linkage algorithm that assumed that microarray data was free of error.
  - A similar failure is reporting of unvalidated results, such as a researcher reporting that linkages have been found, but which are biologically implausible.
  - There is a detailed understanding of biological roles of the components of the human genome, and this domain knowledge should be part of the interpretation of the results.

LIMITS

- ***Limits***:
  - Data is noisy; values may be incorrect or uncertain, and the amount of noise in biological experiments can vary from one laboratory to another.
  - There are inherent biases, the tendency of datasets to consist of very large samples from the same environment. The generality will be lost in this case.
  - The incidence of a particular condition may be very low, and some machine learning techniques are poor with highly imbalanced samples.
  - Knowledge of the extent of such confounds in the data is needed to help assess the significance of the results, and to then, as far as possible, rectify the data.
  - This may involve, for example, careful manual data processing, following explicit guidelines.

RESULTS

- ***Results***:
  - With a good understanding of the data that is to be used, the researcher should be able to make some predictions about the results, or about their form.
  - On an assumption that the method works, the researcher could perhaps estimate the likelihood that a particular level of statistical significance is observed for a particular strength of linkage;
  - or could estimate the size of the smallest data set in which the linkage would be detected.

HUMAN ANNOTATION

- Some experiments depend on **human annotation of data**, to provide a *gold standard* or *ground truth.*
- In document classification, for example, human annotation may indicate the topic of a document: politics, entertainment, sport, and so on.
- In some cases, gathering of annotations can be the dominant cost of an experiment—a factor that is often overlooked, or underestimated, by inexperienced researchers.

PERSUASIVE DATA

- In the process of developing new algorithms, researchers typically use as a testbed a data set with which they are familiar.
- If the algorithm is parameterized in some way **this testbed can be used for tuning**, that is, to identify the parameter values that give the best performance.
- What this tuning process almost certainly cannot do is identify the best parameters for all data sets.
- It is for this reason that descriptions of the research cycle distinguish between an observation phase (used to learn about the object under study) and a testing or confirmation phase (used to validate hypotheses).
- If parameters have been derived by tuning, **the only way to establish their validity is to see if they give good behavior on other data.**

PERSUASIVE DATA

- The research in some fields is underpinned by the availability and use of **reference data sets**.
- Such resources:
  - can be dramatically larger and more comprehensive than the materials that could be created by a typical research team,
  - are easy to explain to readers,
  - and, in principle, allow the direct comparison of work between institutions and between papers.
- However, use of such data also carries risks, in particular of overfitting; that is, methods can become so specialized that they do not work on other data.

PERSUASIVE DATA

- When considering what experiments to try**, identify the data or input for which the hypothesis is least likely to hold**.
- These are the interesting cases: if they are not tested—if only the cases where the hypothesis is most likely to hold are tested then the experiments won't prove much at all.
- The experiment should of course be a test of the hypothesis; you need to verify that what you are testing is what you intended to test.

PERSUASIVE DATA

- Sometimes appropriate **data can be artificial**, or simulated;
- Such data can allow a thorough exploration of the properties of an algorithm.
- But such data should not be used without a clear understanding of its limitations.
  - For example, application of a new hash function to random data is unlikely to be a convincing demonstration that the function is uniform, since the data was uniform to begin with.
- Fundamentally, any scheme for generating artificial data relies on a model, which embodies assumptions and, probably, simplifications.
- The strongest defense of artificial data is to validate it against real data.

INTERPRETATION

- When checking experimental design or outcomes, consider whether there are **other possible interpretations of the results**; and, if so, design further tests to eliminate these possibilities.
- Consider for instance the problem of finding whether a file stored on disk contains a given string.
  - One algorithm directly scans the file; another algorithm, which has been found to give faster response, scans a compressed form of the file.
  - Further tests would be needed to identify whether the speed gain was because the second algorithm used fewer machine cycles or because the compressed file was fetched more quickly from disk.

INTERPRETATION

- Conclusions should be sufficiently supported by the results.
  - Success in a special case does not prove success in general, so be aware of factors in the test that may make it special.
  - A common problem is scale—whether the same result would be observed with a larger data set, for example.
- Don’t draw undue conclusions or inferences.
  - If, say, one method is faster than another on a large data set, and they are of the same speed on a medium data set,
  - that does not imply that the second is faster on a small data set;
  - it only implies that different costs dominate at different scales.

INTERPRETATION

- Another aspect of interpretation is that numerical measures allow numerical manipulation, but such manipulation does not always make sense if applied to the qualitative goal we wish to achieve—the goal may not behave in a numerical way.
- One system may achieve a 20 % higher score than another under some measure of user satisfaction, but it makes little sense to say that the user is 20 % more satisfied.

INTERPRETATION

- A key concept here is of ***predictivity***.
- The main reason that we experiment and measure is to provide evidence about the behavior of a system in general—not just on some specific data set.
- That is, we use measurements on the data we have to hand to make predictive claims about what will happen in the future, when the same system is applied to new data;
- The conclusions in our papers are usually about properties of systems, not their behavior on the data we have already seen.

INTERPRETATION

- Some measures are more predictive than others, however. For example:
  - We could measure a system for translating text between languages by comparing automatic translations to human translations, and counting how many words the automatic and human translations have in common.
  - Alternatively we could rate the automatic translation by how many characters it has—the closer it is in length to the human translation, the better.
  - The method based on words in common should be reasonably predictive: if system A is 30% better than system B on 1,000 sample translations, then we could reasonably expect A to have better commonality-based scores than B on the next 1,000 translations.
  - But suppose that B was better than A according to the length-based measure on the samples. In all likelihood, we would not expect this to predict length-based performance on the new translations;

ROBUSTNESS

- Experiments should as far as possible be **independent of the accuracy of measurements** or quality of the implementation.
- Ideally an experiment should be designed to yield a result that is unambiguously either true or false; where this is not possible, another form of confirmation is to demonstrate a trend or pattern of behavior.
- A simple example: The behavior of query evaluation on a database system with and without indexes.
  - For a small database, the most efficient solution is exhaustive search, because use of an index involves access to auxiliary structures and does not greatly reduce the cost of accessing the data.
  - As database size grows, the cost of data access grows linearly, while index access costs may be more or less fixed.
  - Thus the hypothesis “indexes reduce search costs in large databases” can be confirmed by experiments measuring search costs with and without indexes over a range of database sizes.
  - The trend is independent of the machine and data.

ROBUSTNESS

- Success or otherwise should as far as possible be obvious, not subject to interpretation.
  - If one team develops a piece of software more quickly than does another, is that because (as a researcher might have hypothesized) the first team used a new software development methodology? Or is it because the first team is larger; or smaller; or happier; …
- Results may include some anomalies or peculiarities. These should be explained or at least discussed.
  - Don’t discard anomalies unless you are certain they are irrelevant;
  - They may represent problems you haven’t considered.

EXPERIMENTAL DESIGN EXAMPLE

- As an example of experimental design, consider potential approaches to assessment of the performance of algorithms.
- **Basis of evaluation**.
  - The basis of evaluation should be made explicit.
  - Where algorithms are being compared, specify not only the environment but also the criteria used for comparison.
  - For example, are the algorithms being compared for functionality or speed? Is speed to be examined asymptotically or for typical data? Is the data real or synthetic?
  - When describing the performance of a new technique, it is helpful to compare it to a well-known standard.

EXPERIMENTAL DESIGN EXAMPLE

- The criteria used for comparison can be the principal resources used by algorithms, such as:
  - **Processing time**. Time is not always easy to measure, since it depends on factors such as CPU speed, cache sizes, system load, and hardware dependencies such as prefetch strategy. Times based on a mathematical model rather than on experiment should be clearly indicated as such.
  - **Memory and disk requirements**. It is often possible to trade memory requirements against time, not only by choice of algorithm but also by changing the way disk is used and memory is accessed. You should take care to specify how your algorithms use memory.
  - **Disk and network traffic**. Disk costs have two components, the time to fetch the first bit of requested data (seek and latency time) and the time required to transmit the requested data (at a transfer rate). Thus sequential accesses and random accesses have greatly different costs. The behavior of network traffic is similar—the cost of transmitting the first byte is greater than the cost for subsequent bytes, for example.

HUMAN STUDIES

- A variable in many studies is **the user**.
- Humans need to be involved to resolve many kinds of research question:
  - whether the compressed image is of satisfactory quality,
  - whether the list of responses from the search engine is useful,
  - whether a programming language feature is of value.
- Humans can be used to assess outputs—did the robot do a good job of the housework? Is this Web page suitable for children?
- And humans can be the subject of experiments—what are the main shortcomings of this user interface? Which of these technologies is most helpful for navigating an unfamiliar city?

HUMAN STUDIES

- Appropriate use of humans in experiments allows many forms of rich measurement that provide insights into the value of computational methods.
  - These can be qualitative, such as assessment of ease of use;
  - and subjective, such as self-reported feedback on new technologies.
  - They can also be quantified, through independent observation of behaviors and responses.
- Researchers should consider whether humans are needed for their work, because of the depth and value that a well-designed human intervention can add to measurement of an experiment.
- Design of human studies is treated in detail in research methods texts written for information systems, psychology, or business methods.

HUMAN STUDIES EXAMPLE

- A student developed a tool for identifying and prioritizing changes between versions of legal documents.
- He tested his system **quantitatively**, by creating thirty or so pairs of documents (each pair consisting of an original and a revision of it), and observing how many of 15 readers could find the changes in each pair, either with the new tool or with an existing approach.
- He also tested it **qualitatively**, by interviewing three pairs of authors who used it for some weeks as they collaboratively authored three documents.

DESCRIBING EXPERIMENTS

- Your interpretation and understanding of the results can be as important as the results themselves.
- When describing the outcomes of an experiment, don’t just compile dry lists of figures or a sequence of graphs.
- Analyze the results and explain their significance, select typical results and explain why they are typical, theorize about anomalies, show why the results confirm or disprove the hypothesis, and make the results interesting.

DESCRIBING EXPERIMENTS

- Experiments are only valuable if they are carefully described.
- The description should reflect the care taken—it should be clear to the reader that possible problems were considered and addressed, and that the experiments do indeed provide confirmation (or otherwise) of the hypothesis.
- A key principle is that the experiment should be **verifiable** and **reproducible**.
- Results are valueless if they cannot be repeated by other researchers.
- The description, of both hypothesis and experiment, should be in sufficient detail to allow some form of replication by others. The alternative is a result that cannot be trusted.

DESCRIBING EXPERIMENTS

- Researchers **must decide which results to report**.
  - As discussed earlier, researchers should have logs of experiments recording their history, including design decisions and false trails as well as the results, but such logs usually contain much material of no interest to others.
  - And some results are anomalous—the product of experimental error or freak event—and thus not relevant.
  - But reported results should be a fair reflection of the experiment's outcomes.

DESCRIBING EXPERIMENTS

- If a test fails on some data sets and succeeds on others, it is unethical to conceal the failures, and the existence of failure should be stated as prominently as that of success.
- The experimental outcomes reported in a paper may represent only a fraction of the work that was undertaken in a research program.
  - There will have been exploratory stages and different kinds of failures, and the reported runs may well be carefully chosen to represent a broad range of experiments.
  - Thus the published record of the work is highly selective.

DESCRIBING EXPERIMENTS

- Part of reporting of experiments is **description of the data** that was used.
- Typically, readers need to know:
  - how the data was gathered or created;
  - how your version of the data might be obtained, or recreated;
  - what the shortcomings of the data are, that is in what ways it might be uncertain, incomplete, or unreliable;
  - and what aspects of the research question are not tested by the data.

KEEP RESEARCHERS HONEST

- Researchers are expected to **keep detailed notebooks**. They are invaluable.
- They can be used to record versions and locations of software, parameters used in a particular experiment, data used as input (or the filenames of the data), logs of output (or the filenames of the logs), interpretations of results, minutes of decisions and agreed actions, and so on.
- They allow **easy reconstruction of old research**, and simplify the process of write-up.
- They are particularly helpful if a paper is accepted after a long reviewing process and experiments have to be freshly run; all too often the code no longer produces anything like the original results, because too many details of the experiments have been lost and the code has been modified.
- Most of all, notebooks keep researchers honest.

KEEP RESEARCHERS HONEST

- Researchers should **make code and data available online**.
- Doing so shows that you have high confidence in the correctness of your claims.
- More positively, publishing code reduces the barrier to entry for other researchers, and helps to establish baselines against which new work should be measured.

2. **YOUR RESEARCH CODE**

AS A RESEARCHER

- **Remember**: Software is **not** your product. It is a bridge leading to your experimental results.
- Your product is **knowledge**:
  - New algorithms
  - New theorems
  - New models
  - New explanations of systems
  - New empirical characterization of existing algorithms

CODE IS A MEANS TO AN END

- Optimize for **your time**
  - Except when your research requires your code to be efficient (this happens)
  - Efficient code can be a competitive advantage.
- Think of documentation as notes to yourself
  - Push as much as documentation as possible into identifier names.
- Always ask why you are writing the code...
  - How will this fit into the eventual paper?

CODE IS A MEANS TO AN END

- Do not think that you know now where you will end up.
- Optimize also for **flexibility**.
- Perhaps only design pattern necessary is the strategy pattern
  - Wherever possible, think about where you might want to be clever later.
  - Design for your later cleverness.
- You care about bugs that affect your experimental results, not really about other bugs.

UNLESS YOU SUCCEED

- Commit to reproducible research.
  - If you run an experiment in Sept 2015 you should be able to reproduce the results in Sept 2016
- **You must use version control**!
  - Many examples: RCS, SCCS, CVS, SVN, hg, git, arch
  - Version control everything you care about:
    - Code
    - Research publications
    - Website
- Organization of files, directories.
- “Connect" your code to the figures in the paper.

KEEP TRACK OF PARAMETERS

- Your experiments will have many parameters to tune
  - Which schedule algorithm you used
  - Max queue size
  - Which heuristics for *X*, *Y* , *Z*
  - Learning rate
- Make these parameters of the “*experiment running scripts*".
- Keep track of them in an organized way.

CONNECT PAPER TO RESULT DIRECTORIES

- You need a system where
  - if you open up your paper file a year later
  - you will know how to rerun all the code needed for Figure 3
- Key idea: Generate figures automatically via script
  - Then you just need a map Figure => script file
  - Could use text file
  - Comments in LateX code
- Also need the version of your code.

Principles of EXPERIMENT RUNNING SCRIPTS

1. Automate as much as possible.
   - Lots of times you will need to run a “*parameter sweep*“
   - Helpful to have a “*driver script*" that runs through the parameter grid
   - Interacts well with grid/cloud computing:
     - Run every parameter setting in parallel.
2. Record as much as possible.

TESTING AND TRUST YOUR RESULTS

- Unit tests are great
  - Test a small part of your code.
  - The test code itself checks success, not human.
  - They should be fast to run, so you can run them after every major change.
- Integration test, system test, acceptance test, and failure test;
- Testing and debugging research code can require ingenuity

TESTING

- Always keep the requirements of the customer in mind and make sure they're traceable and identifiable during all levels of testing.
- Before you start testing, create an outline of how the tests will be conducted.
- Consider applying the Pareto principle (rule of 80 % results / 20 % causes) to software testing, meaning 80% of all errors that are identified during testing will likely be traced to 20% of all program modules.
- When you start testing, begin "in the small" and continue until you progress into testing "in the large."
- Remember that it's simply impossible to test all data combinations–your energy should be spent elsewhere.
- Involve an independent third party if necessary, since testing should be as effective as possible.

[1] Mara Calvello, https://fellow.app/blog/engineering/the-levels-of-testing-in-software-engineering-explained/

CODING GUIDELINES

- One task, one tool: decompose the problem into separate pieces of code. In most cases, trying to create a single piece of code that does everything is just not productive.
- Be aware that you may need to trade ease of implementation against realism of the result.
- Use the right tool, not the most convenient tool. For example, if you plan to measure algorithmic efficiency, implement in a suitable language.
- Don’t re-code unnecessarily. Use libraries; is it really necessary to do a fresh implementation of a B-tree?
- For long-running processes, consider developing mechanisms for periodically saving state, so that weeks (or more) of work isn’t lost.

NOTES

- Your code is an investment of your time
  - **Too little**: Can’t trust results, can cause critical error.
  - **Too much**: You are lost or your code is too complicated.
- Follow the code convention of your university / company.
- You need to find the balance between the quality of your computer program and the time of your coding. That balance point must be improved.
- Writing efficient code can give you a competitive advantage as a researcher.

REFERENCES

- Zobel, J. (2014). *Writing for computer science*. London: Springer.
- 2011 Charles Sutton, You and Your Code, Univ. of Edinburgh.